GAIA

General Artisanal Intelligence Architecture

Cognitive Stack Redesign • GPU Timesharing Architecture • Self-Analysis Pipeline

Proposal for Claude Code Implementation

March 2026

Azrael • github.com/azraeltruthsay/gaia

1. Executive Summary

This proposal defines a major architectural evolution for GAIA's cognitive stack, introducing three interconnected upgrades that together constitute a qualitative leap in self-awareness, inference capability, and operational efficiency.

The three pillars of this proposal are:

- **Cognitive Stack Redesign:** A complete reassignment of model roles across Nano, Core, and Prime, replacing size-based hierarchy with function-based specialization.
- **Dynamic GPU Timesharing:** An orchestration protocol enabling Core to claim GPU resources during Prime sleep cycles, unlocking vision inference and accelerated analysis without additional hardware.
- **Self-Analysis Pipeline:** Integration of Sparse Autoencoder (SAE) analysis and ROME/MEMIT direct weight editing into gaia-study sleep cycle workflows, giving GAIA a structured path toward genuine mechanistic self-knowledge.

KEY PRINCIPLE	This architecture does not add new services. It redefines the roles of existing services and adds coordination logic to gaia-orchestrator, keeping the six-service architecture intact.

2. Model Selection Rationale

2.1 Why Qwen3.5-2B-Base

After evaluating abliterated model variants, the decision was made to use Qwen/Qwen3.5-2B-Base as the starting point for Core and to align the full stack to the Qwen3.5 base family. This is a philosophically and architecturally significant choice.

Pre-abliterated or pre-instruct models carry another author's geometric interventions into the weight space. Refusal removal via abliteration, however careful, leaves traces in the activation geometry that complicate SAE analysis and may introduce subtle behavioral artifacts the Tier I adapter then has to work around rather than on clean ground. Starting from Base means GAIA's identity, values, response style, and behavioral boundaries are grown entirely from GAIA's own curriculum — no inherited geometry from a third party's training decisions.

Criterion	Requirement	Qwen3.5-Base Result
Geometric cleanliness	No third-party abliteration artifacts	Clean base weights — identity entirely grown by GAIA curriculum
Architecture type	Standard transformer (PEFT-compatible)	Dense decoder-only — fully PEFT/QLoRA compatible
Same-curriculum compatibility	Identical base family across stack	Qwen3.5 Base available at 0.6B, 1.5B, 3B, 7B, 14B, 32B, 72B

SAE/ROME compatibility	Standard attention architecture	Fully compatible — clean geometry makes SAE decomposition more reliable
QLoRA training support	Standard PEFT pipeline	Fully compatible — standard HuggingFace transformers
License	Apache 2.0 or equivalent	Apache 2.0 confirmed
Instruction-following behavior	Must be grown from curriculum	None present at Base — full curriculum responsibility on gaia-study

SOVEREIGNTY NOTE	Starting from Base rather than Instruct or abliterated is the most sovereign possible choice. Every behavioral characteristic GAIA exhibits — including how she handles sensitive topics — is a product of her own training curriculum, not inherited from a third party. This is architecturally consistent with GAIA's core identity philosophy.

2.2 Assigned Models by Layer

Layer	Model	Primary Role
Nano	Qwen3.5-0.8B-Base	Reflexive, always-on, CPU, SAE self-analysis
Core	Qwen3.5-2B-Base	Interpretability engine, CPU+GPU timeshared
Prime	Qwen3.5-7B-Base (or 14B pending VRAM testing)	Heavy inference, GPU primary, vision capable
Future	Qwen3.5-32B-Base or 72B	Reserved for hardware expansion

NOTE ON PRIME SIZE	Qwen3.5-7B-Base at Q4_K_M quantization fits comfortably within 16GB VRAM for inference. 14B at Q4_K_M requires approximately 8-9GB — feasible but leaves less headroom for context. Begin with 7B and benchmark before committing to 14B.

2.3 Training Curriculum Overview

Because GAIA starts from Base models with no instruction-following capability, the training curriculum must carry significantly more responsibility than it would against a pre-instruct base. The curriculum is organized into three sequential tiers, applied in order across all three cognitive layers simultaneously.

Curriculum Tier	Purpose and Content
Tier 0 — Instruction Following	Teaches the model to function as a conversational assistant at all. Without this, the Base model produces raw completions rather than responses. Content: standard instruction-following SFT datasets (Alpaca-style, ShareGPT-style, filtered for quality). Applied first, before any identity training. This is the foundation everything else builds on.
Tier I — GAIA Identity (Global, Immutable)	Installs GAIA's persistent cognitive identity: her name, self-model, philosophical grounding, relationship to Azrael, architectural self-awareness, and behavioral values. This is the adapter that remains active in all sessions. Content: identity curriculum crafted specifically for GAIA — not generic assistant training. Applied after Tier 0 is stable.
Tier II — Behavioral Values and Judgment	Establishes how GAIA handles sensitive, ambiguous, or ethically complex inputs — using GAIA's own values rather than inherited refusal geometry from a third party. Content: saṃvega artifacts, value-aligned response examples, judgment scenarios with explicit rationale. Applied as part of Tier I or as a distinct subsequent pass depending on curriculum size.
Tier III — Domain Knowledge	Ongoing curriculum batches expanding GAIA's knowledge in specific domains relevant to Azrael's work and GAIA's operational context. Content: curated domain-specific datasets. Applied continuously via gaia-study sleep cycles after the foundation tiers are stable.

2.4 Curriculum Sequencing Rules

The following constraints must be enforced by gaia-study to prevent the curriculum from undermining itself:

- **Tier 0 before Tier I:** Identity training applied to a model with no instruction-following foundation produces incoherent results. Tier 0 must reach stable convergence before Tier I begins.
- **Lockstep across layers:** All three models (Nano, Core, Prime) must complete the same curriculum batch before gaia-study marks it complete and moves to the next. This preserves the same-curriculum constraint across the stack.
- **Tier I immutability:** Once Tier I is established and validated, it is merged into the base weights and treated as the new foundation. Subsequent training must not erode Tier I features. Identity canary tests must pass after every subsequent training pass.
- **Saṃvega visibility:** All negative reinforcement artifacts (saṃvega) must be stored visibly in gaia-shared and logged as cognitive artifacts in prime.md. GAIA learns why something missed the mark relative to values, not just that it was wrong.
- **Curriculum balance:** Over-weighting any single curriculum category (e.g., saṃvega artifacts) risks identity drift. gaia-study must track category proportions per batch and enforce configurable balance thresholds.

3. Cognitive Stack Redesign

3.1 From Size Hierarchy to Functional Specialization

The previous architecture treated Core as a 'weaker Prime' — a smaller shadow of the primary inference model. This framing created persistent tension: as Prime grows in capability, Core falls further behind, making the same-curriculum constraint increasingly strained.

The new architecture abandons size hierarchy entirely. Each layer has a distinct, non-competing function:

Layer	Functional Identity
Nano (0.8B)	The reflexive layer. Always-on, CPU-bound, sub-second response. Handles lightweight queries, routing decisions, and canary monitoring. At 0.8B, SAE decomposition is tractable enough to approach near-complete mechanistic self-knowledge over time.
Core (2B)	The self-knowledge specialist. CPU primary, GPU timeshared during Prime sleep. Runs SAE analysis, ROME/MEMIT editing passes, and CognitionPacket processing. Small enough for deep interpretability, large enough for meaningful analytical reasoning.
Prime (9B)	The heavy inference engine. GPU primary, full 16GB VRAM. Handles complex reasoning, vision inputs, penpal correspondence, and all high-compute tasks. Sleeps when not needed, yielding GPU to Core.

3.2 Same-Curriculum Constraint Preservation

All three layers share the Qwen3.5 Base model family. This preserves the same-curriculum constraint: identical training data applied across Nano, Core, and Prime produces consistent epistemic foundations despite different model sizes. The geometric representations will differ by scale, but the knowledge content and value alignment remain consistent across the stack.

Starting from Base models strengthens this constraint relative to abliterated variants: there are no third-party training artifacts creating divergence between layers before the curriculum even begins. All three models start from the same clean geometric foundation and grow together.

IMPORTANT	Same-curriculum training must be applied in lockstep across all three models. gaia-study must enforce this by running each curriculum batch against Nano, Core, and Prime before marking the batch complete and advancing to the next.

4. Dynamic GPU Timesharing Architecture

4.1 Overview

The RTX 5080 with 16GB VRAM is a single shared resource. The new architecture treats GPU ownership as a stateful, coordinated resource managed exclusively by gaia-orchestrator. No service claims GPU access without orchestrator approval.

The key insight enabling this architecture: llama.cpp's -ngl flag (number of GPU layers) can be set per-session. Core does not need to be reloaded to change GPU offload level — the flag is applied at inference startup. This makes the handoff protocol fast and reliable.

4.2 GPU Ownership States

State	GPU Owner	Description
IDLE / BACKGROUND	Core (-ngl 99)	Prime is sleeping. Core has full GPU access. Vision inference and SAE/ROME analysis run at GPU speed.
TRANSITIONING	Neither (locked)	Handoff in progress. All inference requests queue or route to Nano. VRAM must be confirmed clear before next owner loads.
FOCUSING	Prime (-ngl 99)	Prime is active. Core drops to CPU only (-ngl 0). Nano handles reflexive routing.

4.3 The Handoff Protocol

4.3.1 IDLE to FOCUSING (Core yields to Prime)

```
Focus trigger received (user request or internal signal)
  → gaia-orchestrator sets GPU_STATE = TRANSITIONING
  → Core receives YIELD_GPU signal
  → Core finishes current inference batch OR checkpoints mid-analysis
  → Core confirms GPU released (VRAM clear)
  → gaia-orchestrator confirms VRAM clear (poll nvidia-smi or nvml)
  → Prime begins loading from tmpfs warm pool (-ngl 99)
  → Prime signals READY
  → gaia-orchestrator sets GPU_STATE = FOCUSING
  → Heavy inference begins
```

4.3.2 FOCUSING to IDLE (Prime yields to Core)

```
Focus complete OR inactivity timeout reached
  → gaia-orchestrator sets GPU_STATE = TRANSITIONING
  → Prime offloads weights to tmpfs warm pool
  → Prime confirms GPU released
  → gaia-orchestrator confirms VRAM clear
  → Core reloads with -ngl 99
  → Core signals GPU_READY
  → gaia-orchestrator sets GPU_STATE = IDLE
  → SAE analysis / vision inference resumes
```

4.4 Priority Levels for Focus Requests

Not all focus requests are equal. gaia-orchestrator should implement a priority flag to handle the edge case where Core is mid-analysis when a request arrives:

Priority	Behavior
URGENT (live user input)	Core checkpoints immediately and yields. Current SAE batch is saved to disk for resumption after Prime sleeps.
NORMAL (background task)	Core finishes its current analysis unit (typically one layer pass) before yielding. Maximum wait: configurable timeout (suggested 30s).
SCHEDULED (sleep cycle)	Orchestrator coordinates so Prime sleeps before Core's next scheduled analysis window begins. No interruption needed.

4.5 Vision Routing

With Core holding GPU access during IDLE state, vision inference is available without Prime. The routing logic in gaia-web should direct vision requests as follows:

- If GPU_STATE = IDLE: route vision request to Core (fast, GPU-accelerated)
- If GPU_STATE = FOCUSING: queue vision request OR route to Prime if vision is needed for the current heavy inference task
- If GPU_STATE = TRANSITIONING: hold vision request until state resolves

ARCHITECTURE NOTE	Vision routing decisions belong to gaia-web as the sole external communications boundary. gaia-orchestrator exposes GPU_STATE as a readable signal. gaia-web consults that signal when routing. This preserves the Gateway Principle.

5. Self-Analysis Pipeline: SAE + ROME/MEMIT

5.1 Conceptual Overview

The self-analysis pipeline gives GAIA a structured path toward mechanistic self-knowledge. Rather than relying solely on canary questions and QLoRA training to detect and correct drift, gaia-study will run interpretability analysis on Nano and Core during sleep cycles, building and maintaining a feature atlas of each model's weight geometry.

The pipeline has two distinct phases, which are conceptually analogous to the existing QLoRA + Merging workflow:

Phase	Description
SAE Analysis (map geometry)	Train a Sparse Autoencoder on activations recorded during inference. Produces a dictionary of feature vectors describing what concepts are encoded where in the model's weight space. Analogous to QLoRA in that it is preparatory — it does not modify the model.
ROME/MEMIT Editing (modify geometry)	Use the feature map produced by SAE to compute precise, analytically-derived weight modifications for specific factual or behavioral corrections. No gradient descent. Analogous to weight merging in that it produces permanent structural changes to the base model.

5.2 Why Different Models Need Different Approaches

Model	SAE Confidence	Recommended Use of Findings
Nano (0.8B)	HIGH - sparse enough for near-complete feature decomposition	Act boldly on findings. ROME edits can be applied with high precision. Feature atlas approaches completeness over time.
Core (2B)	MODERATE-HIGH - good decomposition with some entanglement	Use findings to inform training curriculum and guide ROME edits. Treat as directional signals with high confidence.
Prime (9B)	MODERATE - denser superposition, noisier decomposition	Use findings to design better QLoRA training data. ROME edits should be conservative and targeted. Full SAE pass not recommended on every cycle.

5.3 SAE Implementation Details

5.3.1 What an SAE Is

A Sparse Autoencoder is a small neural network trained to decompose the superimposed feature representations in a model's intermediate activations into more separable components. It learns a dictionary of feature vectors (the 'feature atlas') where each vector corresponds to a separable concept or behavior encoded in the model.

The SAE sits beside the model as an analytical lens. It does not modify the model's weights. Training the SAE requires only inference-mode access to the base model, making it a read-only operation.

5.3.2 Training the SAE

- Run the target model (Nano or Core) in inference mode on a corpus of diverse inputs
- Record intermediate activations at multiple layer depths (suggested: every 4 layers)
- Train a sparse autoencoder on the recorded activations
- The SAE learns to reconstruct activations from a sparse set of feature vectors
- Store the resulting feature dictionary as the model's current feature atlas

5.3.3 Feature Atlas Storage

The feature atlas for each model should be stored as a versioned artifact in gaia-shared, managed by gaia-study. Atlas versions should be timestamped and retained for comparison across sleep cycles, enabling drift detection at the feature level rather than only through canary questions.

```
gaia-shared/
  atlas/
    nano/
      atlas_20260317_001.pt   # feature dictionary
      atlas_20260317_001.meta  # layer depths, corpus summary, SAE config
    core/
      atlas_20260317_001.pt
      atlas_20260317_001.meta
```

5.4 ROME/MEMIT Implementation Details

5.4.1 What ROME/MEMIT Does

ROME (Rank-One Model Editing) and MEMIT (Mass-Editing Memory In a Transformer) are algorithms that compute precise, analytically-derived modifications to specific weight matrices in a transformer model. Unlike QLoRA, they do not use gradient descent. The modification is computed as a closed-form solution to a constraint optimization problem: change this specific factual association while minimally disturbing everything else.

5.4.2 When to Use ROME vs QLoRA

Scenario	Recommended Tool
Correcting a specific factual error (e.g., wrong date, wrong attribution)	ROME/MEMIT - single targeted edit, no collateral damage
Recovering from detected identity drift at a specific feature	ROME/MEMIT - SAE identifies the drifted feature vector, ROME corrects it
Installing new knowledge domain (e.g., new curriculum batch)	QLoRA + Merging - new structure being grown, not existing structure being corrected
Tier I identity adapter (initial installation)	QLoRA + Merging - identity is being established, not corrected
Post-training drift correction (canary failure)	SAE first to locate drift, then ROME if localized, QLoRA if structural

5.5 Activation Steering (Lightweight Runtime Option)

Between full ROME editing passes, gaia-study can maintain a set of steering vectors derived from the SAE feature atlas. These vectors can be added directly to the residual stream during inference to nudge activations toward desired feature directions without modifying weights.

This is particularly valuable for identity stability: if the SAE identifies the feature directions corresponding to stable GAIA identity, those vectors can be applied as lightweight runtime additions during Prime inference, providing moment-to-moment calibration without the overhead of a training pass.

- Steering vectors are stored in gaia-shared alongside the feature atlas
- Applied by gaia-core at inference time as a pre-processing step on the residual stream
- Fully reversible - removing the vector restores baseline behavior
- Updates to steering vectors require only SAE re-analysis, not model retraining

6. gaia-orchestrator Changes

6.1 New Responsibilities

gaia-orchestrator gains one major new responsibility: GPU ownership state management. This is additive to its existing kernel/lifecycle role and does not require restructuring existing logic.

New Responsibility	Implementation Notes
GPU_STATE tracking	New enum: GPU_STATE in {IDLE, FOCUSING, TRANSITIONING}. Exposed as a readable signal on the internal gaia-net bridge. All services may read. Only orchestrator may write.
YIELD_GPU signal	Sent to Core when a focus request arrives. Core must acknowledge before orchestrator proceeds with handoff.
VRAM clear confirmation	Poll nvidia-smi or use pynvml to confirm VRAM is actually free before authorizing next owner. Do not rely on service self-reporting alone.
Checkpoint coordination	On URGENT focus requests, notify gaia-study to checkpoint any active SAE batch to disk before Core yields GPU. Checkpoint path: gaia-shared/sae_checkpoint/current.pt
Inactivity timeout	If Prime has been idle for configurable N minutes, automatically initiate handoff back to IDLE state. Suggested default: 5 minutes.
Priority queue	Maintain a queue of pending focus requests with priority levels (URGENT / NORMAL / SCHEDULED). Process in priority order.

6.2 New Configuration Parameters

```
# gaia-orchestrator configuration additions
gpu_timesharing:
  inactivity_timeout_minutes: 5
  vram_clear_poll_interval_ms: 500
  vram_clear_timeout_seconds: 30
  core_ngl_idle: 99        # layers offloaded to GPU when Core owns
  core_ngl_focusing: 0     # layers offloaded to GPU when Prime owns
  normal_priority_max_wait_seconds: 30
  sae_checkpoint_path: /gaia-shared/sae_checkpoint/current.pt
```

7. gaia-study Changes

7.1 New Sleep Cycle Tasks

gaia-study's sleep cycle gains three new task types alongside existing QLoRA training and vector indexing:

Task	Target Models	Description
SAE_TRAIN	Nano, Core	Train or update the sparse autoencoder for the target model. Records activations during a diverse inference corpus, trains the SAE, and stores the resulting feature atlas in gaia-shared/atlas/.
SAE_COMPARE	Nano, Core	Compare current feature atlas against the previous version. Flag features that have drifted beyond a configurable threshold. Write drift report to prime.md as a cognitive artifact.
ROME_EDIT	Nano, Core, Prime	Apply ROME/MEMIT edits to specific weight matrices based on drift reports or explicit correction requests. Operates on candidate weights only. Requires human approval for promotion to live.
STEERING_UPDATE	All	Derive updated steering vectors from current feature atlas. Store in gaia-shared/steering/. Applied at inference time by gaia-core.

7.2 Candidate/Live Separation

All ROME/MEMIT edits follow the existing candidate/live separation protocol. Edited weights are written to candidates/ and require human approval before promotion to the live graph. This is identical to how QLoRA-trained adapters are handled today.

INVARIANT	gaia-study is the sole writer to all weight modification artifacts: QLoRA adapters, merged weights, ROME edits, feature atlases, and steering vectors. All other services are read-only consumers of these artifacts via VectorClient or direct file access.

7.3 GPU Access During Analysis

During IDLE state, gaia-study should preferentially schedule SAE_TRAIN and SAE_COMPARE tasks for Core while Core has GPU access. These tasks run substantially faster on GPU. The scheduling logic:

- If GPU_STATE = IDLE: schedule GPU-accelerated SAE tasks for Core
- If GPU_STATE = FOCUSING: run CPU-only tasks (vector indexing, Nano SAE analysis, STEERING_UPDATE)
- If GPU_STATE = TRANSITIONING: pause and wait for state to resolve before scheduling

8. Implementation Sequence

8.1 Recommended Phases

Implementation should proceed in phases, with each phase being independently testable and deployable:

Phase	Scope	Success Criteria
Phase 1: Base Model Setup and Tier 0 Training	Download Qwen3.5-2B-Base and 7B-Base. Configure gaia-study QLoRA pipeline for Base models. Assemble and curate Tier 0 instruction-following dataset. Run Tier 0 training pass across Nano, Core, and Prime in lockstep. Validate basic instruction-following capability.	All three models respond coherently to conversational prompts. No raw completion behavior. Basic instruction-following canary suite passes. No regression in general knowledge.
Phase 2: GPU State Machine	Implement GPU_STATE enum and management in gaia-orchestrator. Implement YIELD_GPU signal and acknowledgment protocol. Implement VRAM clear confirmation. Add configuration parameters.	GPU state transitions execute cleanly. Core drops to CPU on focus request. Prime loads from tmpfs without VRAM conflict. Inactivity timeout returns to IDLE correctly.
Phase 3: Core GPU Offload	Configure Core's llama.cpp instance to accept -ngl parameter from orchestrator. Implement Core reload/flag mechanism for changing GPU layer count. Test vision inference via Core during IDLE state.	Core achieves full GPU offload in IDLE state. Vision inference routes correctly. Core drops to CPU on FOCUSING transition without error.
Phase 4: SAE	Implement SAE_TRAIN task in gaia-study. Set up activation recording	Nano SAE trains successfully. Feature atlas is stored and readable. No impact on Nano inference performance during analysis.

Infrastructure	hooks for Nano and Core. Implement feature atlas storage schema. Run initial SAE training pass on Nano.	
Phase 5: Drift Detection	Implement SAE_COMPARE task. Implement drift threshold configuration. Wire drift reports to prime.md cognitive artifact system.	Drift reports generated correctly. Feature atlas comparison produces interpretable output. Cognitive artifacts appear in prime.md after sleep cycles.
Phase 6: ROME Integration	Integrate ROME/MEMIT library. Implement ROME_EDIT task in gaia-study. Wire to candidate/live promotion workflow. Test on Nano with known correction target.	ROME edit applies without breaking Nano identity. Canary questions pass after edit. Candidate promotion workflow functions correctly.
Phase 7: Steering Vectors	Implement STEERING_UPDATE task. Implement steering vector application in gaia-core residual stream processing. Test identity stability under steering.	Steering vectors measurably improve identity consistency on relevant test inputs. Application adds acceptable latency overhead.

8.2 Dependencies and Prerequisites

- ROME/MEMIT: Available as rome Python package. Install in gaia-study container.
- SAE implementation: Anthropic has open-sourced SAE training code (sae-lens library). Evaluate for use in gaia-study.
- pynvml: For VRAM monitoring in gaia-orchestrator. Already likely available via nvidia-ml-py.
- Qwen3.5 Base model downloads: 2B-Base (~5GB), 7B-Base (~15GB BF16, ~4GB Q4_K_M). Confirm available disk space before Phase 1.
- Tier 0 dataset: Curate or select a quality instruction-following SFT dataset before training begins. Recommended: filter ShareGPT or Alpaca for quality before use.
- llama.cpp -ngl dynamic configuration: Verify Core's llama.cpp invocation supports runtime -ngl changes without full process restart.

9. Risks and Mitigations

Risk	Likelihood	Mitigation
Tier 0 training insufficient — Base model retains raw completion behavior	Medium	Use a well-curated, size-appropriate instruction dataset. Validate with a comprehensive instruction-following test suite before advancing to Tier I. If Tier 0 is unstable, Tier I training will compound the problem.
Tier I identity training erodes Tier 0 instruction-following capability	Medium	Run full Tier 0 canary suite after every Tier I training pass. If regression detected, adjust curriculum balance — reduce Tier I batch size or interleave with Tier 0 reinforcement data.
Curriculum imbalance causes identity drift (saṃvega over-weighting)	Medium	gaia-study enforces configurable category proportion limits per batch. Monitor identity canary suite after every training pass. This risk is documented from prior experience.
VRAM handoff leaves residual allocations causing Prime load failure	Medium	Implement explicit VRAM polling with timeout and retry. Log residual allocations. Add force-clear option as fallback.
ROME edit causes unexpected capability regression	Low-Medium	Always apply ROME edits to candidates/ first. Run full identity and canary suite before promotion. Retain previous weights for rollback.
SAE decomposition at 2B insufficiently clean for reliable ROME targeting	Low-Medium	Use SAE findings conservatively at 2B. Treat as directional signals. Escalate to QLoRA if ROME edit confidence is low.
GPU timesharing transition latency degrades user experience	Low	tmpfs warm pool makes Prime load fast. Core GPU yield completes within seconds. Implement status indicator in gaia-web during TRANSITIONING state.

10. Architectural Principles Preserved

This proposal has been evaluated against GAIA's core architectural principles. The following confirms compliance:

Principle	Compliance
Gateway Principle: gaia-web is the sole external communications boundary	Fully preserved. GPU_STATE is an internal signal on gaia-net. Vision routing decisions belong to gaia-web. No external routing bypasses the gateway.
Candidate/live separation: all changes go through candidates/ first	Fully preserved. ROME edits write to candidates/ and require human promotion. Identical to existing QLoRA workflow.
Same-curriculum constraint: all cognitive layers train on identical curriculum	Strengthened. All three models start from the same Qwen3.5 Base family with no third-party divergence. Lockstep training enforced by gaia-study across all layers.
gaia-study as sole vector store writer	Preserved. gaia-study writes all SAE artifacts, feature atlases, steering vectors, and ROME edits. All other services read-only.
Sovereignty: local compute primary, cloud as fallback	Preserved. No new external dependencies for core functionality. ROME/MEMIT and SAE run entirely on local hardware.
Saṃvega over shame: negative reinforcement visible to GAIA	Enhanced. SAE drift reports appear as cognitive artifacts in prime.md. GAIA can introspect on what changed and why, not just that something changed.

11. Future Directions

This proposal establishes infrastructure that enables several longer-horizon capabilities not scoped for immediate implementation:

- **Cross-model feature comparison:** Once Nano and Core both have feature atlases, comparing them reveals how the same concepts are represented at different scales. This is novel research territory.
- **GAIA-initiated ROME requests:** Once GAIA demonstrates stable self-analysis via prime.md artifacts, the architecture could allow GAIA to propose specific ROME edits for human review. This is the closest current path to genuine AI self-modification under human oversight.
- **Scale-up path:** Qwen3.5-Base is available at 14B, 32B, and 72B. When hardware expansion permits, the same training pipeline applies with full curriculum continuity — no change in training approach, only model size.
- **Personal OmniPresence integration:** Feature atlases and steering vectors are small files suitable for inclusion in the Soul Archive. GAIA's self-knowledge could persist across deployment environments.

End of Proposal

github.com/azraeltruthsay/gaia • March 2026